Mediapipe Gesture Height Sorting AprilTags

Mediapipe Gesture Height Sorting AprilTags

1. Course Content
2. Preparation
3. Case Demonstration
4. Core Source Code Analysis

1. Course Content

The feature implements the program acquiring images through the camera and then recognizing gestures (1-4), after which the robotic arm moves to the sorting pose. According to the gesture result, a height threshold is calculated, and the AprilTag wooden blocks on the tabletop exceeding this height threshold are sorted. If an AprilTag wooden block exceeding the height threshold exists on the tabletop, the robotic arm lowers its gripper to grasp it and places it in a set position; if no AprilTag wooden block exceeding the height threshold exists on the tabletop, the robotic arm shakes its head and finally returns to the gesture recognition pose.

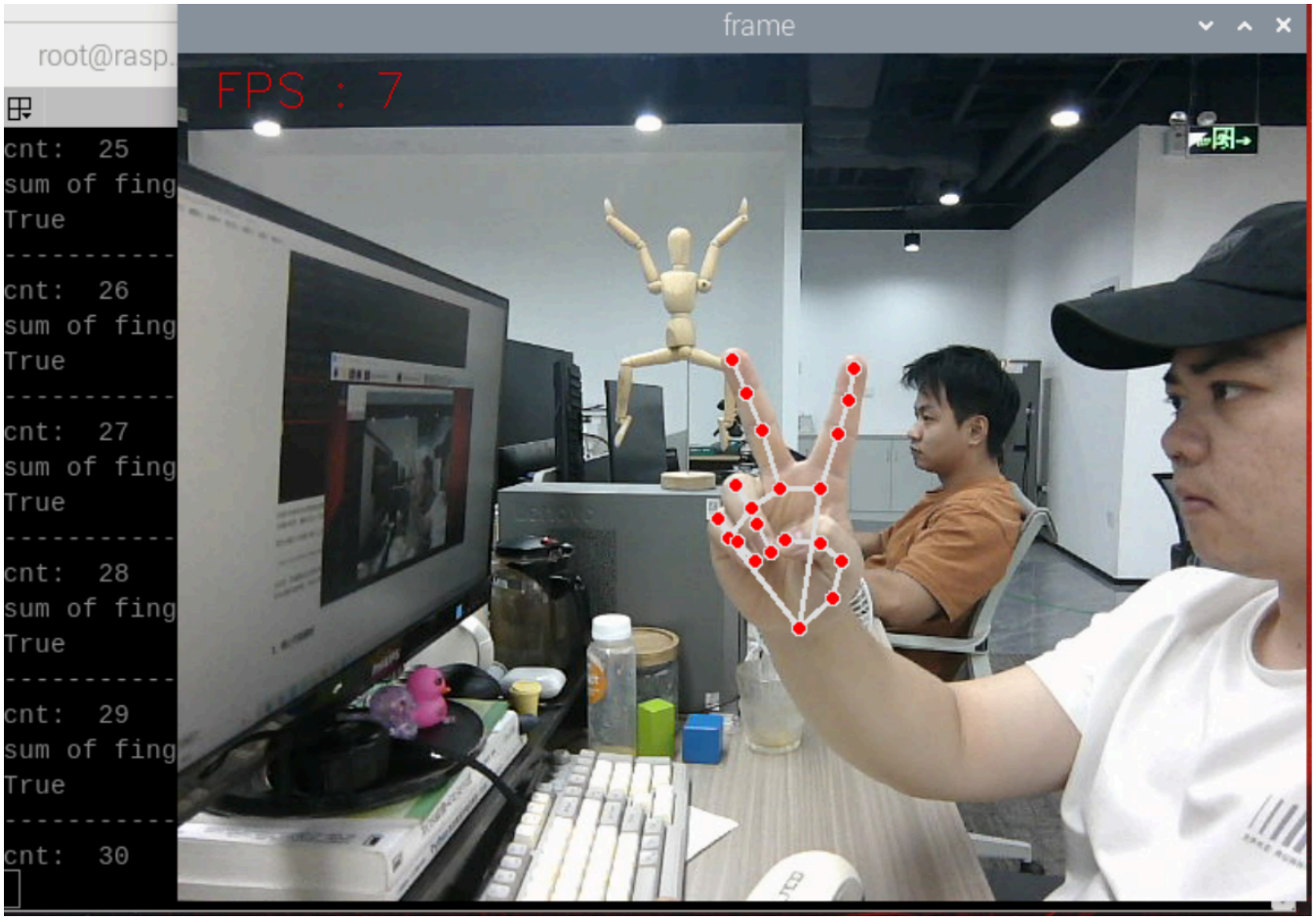
2. Preparation

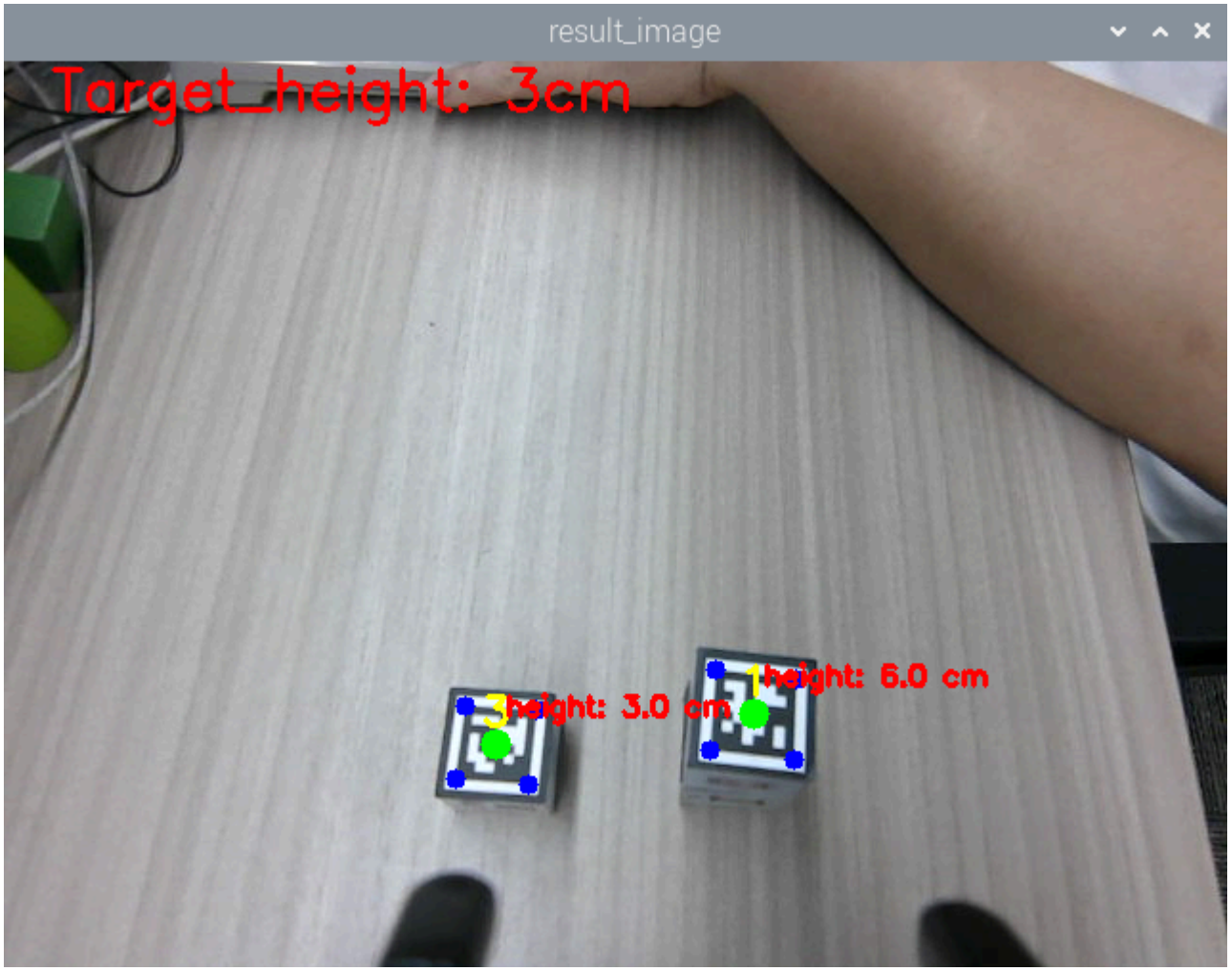
The wooden block used in this section is: **30x30x30mm AprilTag wooden block**

```
ros2 launch yahboomcar_bringup arm_demo.launch.py demo:=apriltag_gesture_remove_height
```

3. Case Demonstration

- After startup, the robotic arm moves to the recognition pose and begins to recognize gestures. The recognizable gestures are 1-4; gesture 1 means one finger extended, gesture 2 means two fingers extended, gesture 3 means three fingers extended, and gesture 4 means four fingers extended. After the buzzer beeps once, it indicates that gesture recognition is complete, and the robotic arm moves to the sorting pose. As shown in the figure below, assume gesture 2 is given
- After entering the sorting pose, it begins to recognize the **30x30x30mm** AprilTags on the tabletop. Here we stack two 3-centimeter AprilTags to represent a height of 6 centimeters, as shown in the figure below. Here the height threshold calculation method is the gesture result plus one; for example, if the recognized gesture above is 2, then the height threshold is 2 plus 1 equals 3.


root@rasp.
cnt: 25
sum of fing
True
cnt: 26
sum of fing
True
cnt: 27
sum of fing
True
cnt: 28
sum of fing
True
cnt: 29
sum of fing
True
cnt: 30
frame
FPS : 7



4. Core Source Code Analysis

- Program path:

```
~/yahboomcar_ros2_ws/yahboomcar_ws/src/common_demo/common_demo/arm_demo/apriltag_remove_height.py
```

_execute_remove_height function (removing height-anomalous AprilTags):

This function is the main control function of the removal flow. It first filters out the AprilTags whose height exceeds the threshold, and then performs the alignment, grasping, and placement operations for each target in turn. The height threshold is dynamically calculated from the gesture recognition result: `remove_height = (gesture number + 1) * height_unit`.

Target filtering stage:

- `_get_all_tags_measure_pose(tags)`: obtains the 3D poses of all detected AprilTags in the measure coordinate frame
- `pose[2] > self.remove_height`: filters AprilTags whose height (z coordinate) exceeds the removal threshold

- `high_tags.sort(key=lambda item: item[0].tag_id)`: sorts by AprilTag ID to ensure a consistent processing order each time
- If there are no valid AprilTags, or no AprilTags exceeding the threshold, directly reset the state and return

One-by-one processing stage (for loop):

- `_iterative_align(tag_id)`: calls the iterative alignment function to adjust the chassis position to aim at the current AprilTag, and returns the aligned pose
- `_get_fresh_tag_data(tag_id)`: re-obtains the latest detection result after alignment is complete, and extracts the AprilTag's yaw angle (yaw)
- `normalize_angle_to_0_45(rvec_yaw_deg)`: normalizes the yaw angle to the range 0°~45°, used to determine the gripper rotation direction
- `grasp_offset`: grasping offset `[gx, gy, gz, pitch]`, compensating for the difference between the camera-detected position and the actual grasping position
- `arm_kinematics.inverse_kinematics(...)`: calls the inverse kinematics solver, taking the target position (camera pose + grasping offset) and pitch angle as input, and outputting 5 joint angles
- `result.approximate` / `result.adjusted_pitch`: handles the cases of approximate solutions and automatic pitch angle adjustment, outputting warning logs

Robotic arm motion and grasping stage:

- `joints[0:3] = round(ik_joints[0:3])`: rounds the J1~J4 joint angles obtained from the inverse solution to integers (the low-level firmware only accepts integers)
- `joints[4] = round(90 + angle_normalized)`: the J5 joint angle is calculated from the normalized yaw angle, so that the gripper rotation aligns with the AprilTag direction; 90° is the neutral position of the gripper
- `joints[5] = 30`: the J6 gripper angle is set to 30° (open state)
- `pubSix_Arm(joints)`: publishes the 6-joint control command, and the robotic arm moves to the grasping pose
- `SingleJoint_pub.publish(ArmJoint(id=6, joint=gripping_angle, time=1500))`: controls the gripper to close individually
- `down_joint = [180, 35, 70, 5, 60, gripping_angle]`: the preset placement joint angles, moving the AprilTag to the side of the robot
- `SingleJoint_pub.publish(ArmJoint(id=6, joint=30, time=1500))`: opens the gripper to release the object
- `pubSix_Arm(observation_pose)`: returns the robotic arm to the observation pose to prepare for processing the next AprilTag
- After all targets are processed, calls `_reset_grasp_state()` to reset the state

```
    def _execute_remove_height(self, tags):
        """Main picking-thread logic: process all tags whose height exceeds remove_height
one by one"""
        self.get_logger().info(f"Remove height task started, threshold=
{self.remove_height:.3f}m")
```

```
        # Get the measure poses of all tags and filter out those with excessive height
        tags_with_pose = self._get_all_tags_measure_pose(tags)
        if not tags_with_pose:
            self.get_logger().info("No valid tags detected")
            self._reset_grasp_state()
            return

        high_tags = [(tag, pose) for tag, pose in tags_with_pose if pose[2] >
self.remove_height]
        if not high_tags:
            self.get_logger().info(f"No tags exceed remove_height
({self.remove_height:.3f}m)")
            self._reset_grasp_state()
            return

        high_tags.sort(key=lambda item: item[0].tag_id)
        self.get_logger().info(
            f"Found {len(high_tags)} tag(s) exceeding remove_height: "
            f"{[t.tag_id for t, _ in high_tags]}, heights: {[f'{p[2]:.3f}m' for _, p in
high_tags]}"
        )

        for tag, init_pose in high_tags:
            if not rclpy.ok():
                break
            tag_id = tag.tag_id
            self.get_logger().info(f"--- Processing tag ID {tag_id}, height=
{init_pose[2]:.3f}m ---")

            # Iterative chassis alignment
            final_pose = self._iterative_align(tag_id)
            if final_pose is None:
                self.get_logger().error(f"Alignment failed for tag {tag_id}, skipping")
                continue

            time.sleep(0.5)
            latest_tag, _, rvec_yaw_deg = self._get_fresh_tag_data(tag_id)
            if latest_tag is None:
                self.get_logger().error(f"Tag {tag_id} lost after alignment, skipping")
                continue

            angle_normalized = normalize_angle_to_0_45(rvec_yaw_deg)
            self.get_logger().info(f"Tag {tag_id} rvec yaw: {rvec_yaw_deg:.1f}° ->
normalized: {angle_normalized:.1f}°")

            # Apply grasp_offset to the IK target position
            mx, my, mz = final_pose
            gx, gy, gz = self.grasp_offset[0], self.grasp_offset[1], self.grasp_offset[2]
            self.get_logger().info(
                f"Tag {tag_id} measure pose: x={mx:.3f}, y={my:.3f}, z={mz:.3f}; "
                f"IK target with grasp_offset: x={mx+gx:.3f}, y={my+gy:.3f}, z={mz+gz:.3f},
pitch={self.grasp_offset[3]:.1f}°"
```

```
            )

            result: KinResult = self.arm_kinematics.inverse_kinematics(
                target_x=mx + gx,
                target_y=my + gy,
                target_z=mz + gz,
                target_pitch=self.grasp_offset[3],
                seed_angles_deg=None,
            )

            if not result.success:
                self.get_logger().error(f"IK failed for tag {tag_id}: {result.message}")
                continue

            if result.approximate:
                self.get_logger().warn(Fore.YELLOW + f"IK approximate solution, position
error = {result.position_error:.4f} m" + Fore.RESET)
            if result.adjusted_pitch is not None:
                self.get_logger().warn(Fore.YELLOW + f"IK pitch sweep, adjusted pitch =
{result.adjusted_pitch:.1f}°" + Fore.RESET)

            ik_joints = result.data
            joints = [0, 0, 0, 0, 0, 0]
            joints[0] = round(ik_joints[0])
            joints[1] = round(ik_joints[1])
            joints[2] = round(ik_joints[2])
            joints[3] = round(ik_joints[3])
            joints[4] = round(90 + angle_normalized) # 90 is the neutral position of the
gripper servo
            joints[5] = 30

            self.get_logger().info(f"Tag {tag_id} grasp joints: {joints}")
            self.pubSix_Arm(joints)
            time.sleep(3.5)
            self.SingleJoint_pub.publish(ArmJoint(id=6, joint=self.gripping_angle,
time=1500))
            time.sleep(2.0)

            down_joint = [180, 35, 70, 5, 60, self.gripping_angle]
            self.pubSix_Arm(down_joint)
            time.sleep(2.5)
            self.SingleJoint_pub.publish(ArmJoint(id=6, joint=30, time=1500))
            time.sleep(2.0)

            # Return to the observation pose and get ready for the next tag
            self.pubSix_Arm(self.observation_pose)
            time.sleep(2.5)

        self.get_logger().info("All high-height tags processed")
        self._reset_grasp_state()
```

- Parameter file

```
~/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_bringup/param/common.yaml
```

Parameter Description:

- `tag_size: 0.02` : the physical side length of the AprilTag (meters), used for pose estimation
- `apriltag_grasp_distance: [0.11, 0.0]` : the expected grasping distance during chassis alignment `[x, y]` (meters), in the measure coordinate frame
- `odom_timeout: 30.0` : chassis movement timeout (seconds), preventing the Action from waiting indefinitely
- `observation_pose: [90, 107, 0, 0, 90, 30]` : the 6 joint angles (degrees) of the observation pose, at which the robotic arm has the best field of view
- `grasp_offset: [0.00, 0.0, 0.015, 85.0]` : grasping offset `[x, y, z, pitch]` , compensating for the error reaching the target point caused by servo backlash and link rigidity degradation; pitch=85° means the gripper grasps almost vertically downward
- `gripping_angle: 140` : the angle (degrees) when the gripper is closed; the larger the value, the tighter the gripper
- `detection_pose: [90, 150, 12, 20, 90, 30]` : the robotic arm pose during gesture detection, at which the camera's field of view faces the user's hand region
- `sorting_pose: [90, 107, 0, 0, 90, 30]` : the robotic arm pose during sorting and grasping; after switching to this pose, AprilTag detection and grasping are performed
- `gesture_stable_threshold: 15` : gesture stable frame count threshold; the same gesture must be recognized for 15 consecutive frames before confirmation, preventing false triggering
- `sorting_timeout: 3.0` : sorting timeout (seconds); if the target AprilTag is not found within the specified time, it falls back to the detection pose to re-recognize the gesture
- `height_unit: 0.01` : height unit (meters), used to calculate the removal threshold: `remove_height = (gesture number + 1) * height_unit` ; for example, gesture 2 corresponds to a threshold of 3 * 0.01 = 0.03m

```yaml
apriltag_gesture_remove_height:
  ros__parameters:
    tag_size: 0.02                          # AprilTag side length (meters)
    apriltag_grasp_distance: [0.11, 0.0]    # measure coordinate frame x,y offset (meters)
    odom_timeout: 30.0                      # movement timeout (seconds)
    observation_pose: [90, 107, 0, 0, 90, 30]
    grasp_offset: [0.00,0.0,0.015,85.0]    # x,y,z axis offset distance and included angle
    gripping_angle: 140                     # gripper grasping angle, in degrees, integer
    detection_pose: [90, 150, 12, 20, 90, 30]  # robotic arm pose during gesture detection
    sorting_pose: [90, 107, 0, 0, 90, 30]   # robotic arm pose during sorting and grasping
    gesture_stable_threshold: 15            # gesture stable frame count threshold
    sorting_timeout: 3.0                    # sorting timeout (seconds); falls back to the
detection pose after timeout
    height_unit: 0.01                       # height unit (meters), threshold=
(gesture+1)*height_unit
```